

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Сибирский государственный университет телекоммуникаций и
информатики»

Кафедра вычислительных систем

Отчет по лабораторной работе №2
по дисциплине
Архитектура распределенных приложений

по теме:
ЗНАКОМСТВО С CI/CD. ДОБАВЛЕНИЕ КОНТЕЙНЕРА DOCKER В
РЕЕСТР GITLAB

Студент:
Группа ИА-331

Я.А Гмыря

Предподаватель:
Преподаватель

Т.В Челканова

Новосибирск 2025 г.

СОДЕРЖАНИЕ

ЦЕЛЬ И ЗАДАЧИ	3
1 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ.....	4
2 НАПИСАНИЕ .GITLAB-CI.YML	7
3 ВЫВОД	11

ЦЕЛЬ И ЗАДАЧИ

Цель: Познакомиться с CI/CD, написать собственный pipeline, который бы пушил в реестр docker-образ после успешной сборки

Задачи:

1. Познакомиться с CI/CD концепцией
2. Написать pipeline

ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Что такое CI/CD?

CI (Continuous Integration)

Автоматизация сборок и тестов. Разработчики часто коммитят свой код, а система CI автоматически запускает сборку и выполняет автоматизированные тесты. Благодаря частым проверкам, ошибки обнаруживаются на ранних стадиях разработки, что значительно упрощает их исправление. Все это повышает конечное качество продукта, ведь проверки выполняются постоянно.

CD (Continuous Delivery)

После успешного завершения CI-этапа код автоматически разворачивается на тестовые или промежуточные среды, где проводится дальнейшая проверка. После всех этапов CI и CD код автоматически разворачивается на продакшн, то есть доступен конечным пользователям.

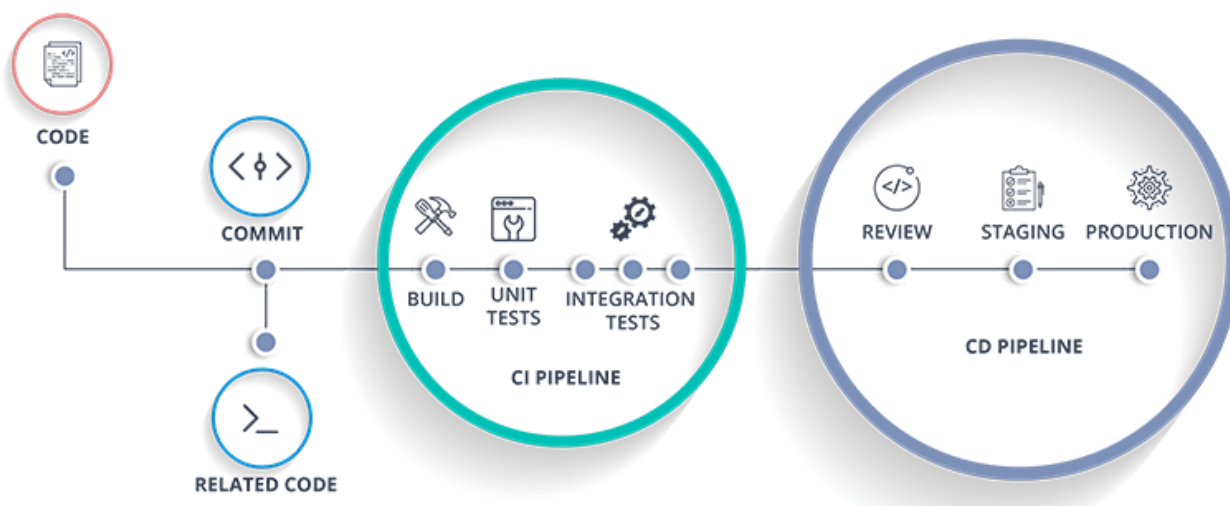


Рисунок 1 — Схема работы CI/CD

Преимущества

1. Ускорение циклов разработки.
2. Повышение качества продукта.

3. Сосредоточение на коде.
4. Надежность.

GitLab Container Registry

Приватное хранилище Docker-образов, связанное с вашим проектом или группой. Он работает примерно как Docker Hub, только интегрирован прямо в GitLab. Поддерживает версии и теги образов. Можно автоматически собирать и пушить образы из pipeline.

buildx

Расширенная версия команды `docker build`. Стала популярной благодаря мультиплатформенной сборке, кешированию сборки (ускоряет сборку при последующих коммитах), возможности пушить во время сборки и параллельной сборке. Иными словами - более продвинутый и удобный вариант сборки контейнеров.

Встроенные переменные CI

Автоматические переменные окружения, доступные в пайплайне, которые позволяют использовать данные о проекте, коммитах, ветках, тегах и реестре. Эти переменные упрощают и ускоряют работу с CI. Программисту больше не нужно задумываться, где ему взять имя ветки, хеш коммита и прочее. Таких переменных очень много. Полный список и описание можно найти на оф.сайте [gitlab](https://docs.gitlab.com/ci/variables/).

Пример переменных:

1. CI_REGISTRY - адрес реестра (допустим `registry.gitlab.com`)
2. CI_REGISTRY_USER - логин для `docker login` (допустим `gitlab ci token`)
3. CI_REGISTRY_PASSWORD - пароль для `docker login` (допустим `glpat-xxxxxx` (автотокен))
4. CI_REGISTRY_IMAGE - путь к образу проекта (допустим `registry.gitlab.com/group/project`)

5. CI COMMIT SHORT SHA - короткий хеш коммита на 7 символов (допустим a1b2c3d)
6. CI COMMIT TAG - релизный тег образа (допустим v1.0.0, если коммит был с тегом)

Как реализуется работа с CI/CD в GitLab?

Pipeline

Набор последовательных шагов, которые автоматически выполняются при изменении кода, т.е, когда код заливается, то триггерется CI pipeline и начинается сборка/тесты и т.д.

CI Pipeline

Реализация CI pipeline должна находиться в специальном файле **.gitlab-ci.yml** в корневой директории проекта. В нем описываются стадии, через которые проходит код при коммите. Происходит это путем создания определенных блоков, в которых описываются инструкции или другие блоки.

НАПИСАНИЕ .GITLAB-CI.YML

План написания файла .gitlab-ci.yml

1. В корневой директории проекта создать файл с названием .gitlab-ci.yml
2. Создать стадии (в нашем случае достаточно одной)
3. Создать джобу, которая будет принадлежать нашей стадии
4. Задать образ контейнера, в котором будет выполняться джоба
5. Задать дополнительный контейнер с демоном докера
6. Перед основным скриптом выполнить авторизацию в хранилище
7. Создать контекст для контейнера с демоном
8. Добавить в контекст сборщик с помощью docker buildx
9. В основном скрипте сформировать логику создания имени образа для тегированного и нетегированного коммита. В случае коммита без тега в имени образа указать короткий хэш коммита, в ином случае - тег
10. Произвести сборку образа для архитектур amd/arm 64

Для создания стадий используется ключевое слово **stages** после которого указываются стадии, которые позже будут показаны в pipeline. Стадий может быть сколько угодно: под сборку, под тесты и т.д в зависимости от задач.

```
stages:  
  - build
```

Но это всего лишь объявление без указания инструкций. Для указания инструкций нужно создать **job**, где нужно указать, имя джобы и к какой стадии она принадлежит (у каждой стадии может быть много джоб).

```
build-and-push:  
  - stage: build
```

Помимо этого можно указать основной контейнер-образ, в котором будет запускаться pipeline (код в .gitlab-ci.yml выполняется НЕ на локальном хосте, а на удаленной машине, и можно выбирать тип контейнера), для этого используется ключевое слово **image**.

```
image: docker
```

Это значит, что наш pipeline будет запущен в контейнере, в котором уже будет установлен docker-клиент.

С помощью ключевого слова **services** задается вспомогательный контейнер, который запускается рядом с контейнером **image**

```
services:  
  - name: docker:24-dind
```

Здесь используется `docker:dind` (Docker in Docker) — отдельный контейнер, где работает Docker-демон. Основной контейнер (`docker:24`) подключается к этому сервису и может запускать сборку образов. Зачем это нужно? Дело в том, что `image:docker` имеет только docker-клиент, но не имеет демона, а демон как раз занимается работой контейнеров (создает, запускает, останавливает и т.д). Иначе можно сказать, что клиент - это API, удобная оболочка над демоном. Dind как раз запускает демона, который и позволяет нам работать с контейнерами.

Для указания самих инструкций, которые будут выполняться в pipeline используется блок **script**

```
script:  
  - TAGS="$CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA"  
  - if [ -n "$CI_COMMIT_TAG" ]; then TAGS="$TAGS -t $CI_REGISTRY_IMAGE:$CI_COMMIT_TAG"; fi  
  - docker buildx build --platform linux/amd64,linux/arm64 -f ./Dockerfile ./ -t $TAGS --push
```

Рисунок 2 — Script block

Также есть блок, который выполняется перед блоком **script** и используется для настройки окружения, авторизации и прочих вспомогательных действий. Блок задается ключевым словом **before_script**.

```
before_script:  
  - echo "$CI_REGISTRY_PASSWORD" | docker login -u "$CI_REGISTRY_USER" --password-stdin "$CI_REGISTRY"  
  - docker context create context  
  - docker buildx create --name mybuilder --driver docker-container --use context
```

Рисунок 3 — Before_script block

Авторизация в реестре образов

Зачем нужна авторизация? Все дело в безопасности: вдруг кто-то посторонний начнет заливать свои образы в частный реестр или вдруг начнет удалять имеющиеся? Для этого и нужна авторизация, чтобы доступ имел ограниченный круг лиц и у этих лиц были ограниченные права (допустим push и pull). Авторизация происходит с помощью команды

```
docker login [OPTIONS] [SERVER]
```

Пример реальной команды:

```
before_script:
- echo "$CI_REGISTRY_PASSWORD" | docker login -u "$CI_REGISTRY_USER" --password-stdin "$CI_REGISTRY"
```

Рисунок 4 — Авторизация в реестре

Здесь ключ `-u` используется для передачи в команду имени пользователя, а `--password-stdin` - для передачи пароля пользователя в потоке `stdin`. В конце указываем имя реестра. Зачем передавать пароль в `stdin`? Это делается с целью безопасности: в таком случае при сборке в логах не всплывет пароль.

Создание контекста и добавление сборщика

Контекст создается для контейнера, запущенного параллельно основному, в котором будет происходить сборка образа. Контекст по сути содержит в себе настройки для этого образа. Далее необходимо задать сборщика `docker buildx` с определенными параметрами и связать сборщик с контекстом.

Пример кода

```
- docker context create context
- docker buildx create --name mybuilder --driver docker-container --use context
```

Рисунок 5 — Создание и настройка контекста

Здесь сначала создаем контекст, а далее создаем билдер. Ключ `--name` задает имя билдера, ключ `--driver` задает, как будет происходить сборка: в локальном демоне `docker` (если указать `docker`) или в BuildKit (если указать `docker-container`). BuildKit позволяет создавать мультиплатформенные сборки, поэтому выбираем его. Далее выбираем, к какому контексту ”прицепить” билдер.

Создание имени образа (тега)

Тег - то наименование образа, которое будет указываться в реестре. Пример формирования тега:

```
script:
  - TAGS="$CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA"
  - if [ -n "$CI_COMMIT_TAG" ]; then TAGS="$TAGS -t $CI_REGISTRY_IMAGE:$CI_COMMIT_TAG"; fi
```

Рисунок 6 — Создание тега

Изначально тег состоит из адреса докер-репозитория и короткого хеша. Если коммит был с тегом, то добавляем в конце тег (допустим v 1.0.0), если нет, то пушим с дефолтным тегом.

Сборка и пуш

Теперь самое главное - собрать образ и запустить. Пример кода:

```
script: - docker buildx build --platform linux/amd64,linux/arm64 -f ./Dockerfile ./
-t TAGS --push
```

Ключ `--platform` нужен для указания архитектур, для которых будет производиться сборка. `-f` нужен для указания пути до Dockerfile. `-t` указывает на тег образа. `--push` нужен для пушинга в реестр при удачной сборке.

Полный код:



Рисунок 7 — Полный код

ВЫВОД

В ходе работы я познакомился с CI/CD - важной концепцией в мире разработки ПО. Написал собственный CI pipeline и попутно познакомился с работой с чатсными реестрами образов.